

CLOUD SQUAD

클라우드 4주차

2025.05.14

BY 김대현

Contents

01	_____	Image Build
02	_____	Container 실행
03	_____	Docker Volume

외부 공유 금지!!!
카카오 내부 자료도 있기 때문에....

Docker Image build

01 Image Build

Image?

- Docker 공식문서: <https://docs.docker.com/reference/cli/docker/buildx/build/>
 - **Image는 컨테이너를 생성하기 위해 필요한 모든 것을 포함한 파일 묶음**
 - 서버 실행 소스코드, 명령어, 옵션 정보등을 포함
 - **image는 파일이기 때문에 네트워크를 통해서 쉽게 주고 받을수 있음** → 단, 정확히 어떤 이미지 사용했는지 알아야함
 - 또한 이미지는 버전관리 시스템을 가짐: Digest (다이제스트)
1. **Digest (다이제스트) → 자동으로 생성되는 해시값. 그래서 별칭을 정함 (태그)**
 - 태그는 별칭이기 때문에 시점에 따라 다른 Digest 참고 → 보통 최신 이미지 태그 (latest, 최신 다이제스트)
 2. **저장 효율 → 이미지 크기가 커지면 네트워크를 통해 주고 받을때 시간이 오래 걸릴수도.**
 - a. 그래서 Layer 방식으로 저장 → **이미지를 Layer 단위의 파일로 나눠서 저장**
 - b. 그렇게 하면 **같은 내용을 중복 저장 방지, 저장 효율을 높이기 위해 사용**

01 Image Build

Image Build

docker buildx build 사용법

- -t <레포>:<태그>: 이미지에 이름과 태그를 붙입니다.
- --build-arg: Dockerfile 내 ARG로 선언된 변수에 값을 전달합니다.
- -f: Dockerfile 위치 지정. 생략 시 기본 경로(현재 디렉토리의 Dockerfile)를 사용합니다.
- --pull: 베이스 이미지를 항상 최신으로 당겨옵니다.

```
docker buildx build \  
> -t toby/mbti:embedded-db \  
> -t toby/mbti \  
> --build-arg NODE_VERSION=20.15.1 \  
> -f ./Dockerfile \  
> --pull \  
> .
```


01 Image Build

Image Build

- `-t toby/mbti:embedded-db`와 `t toby/mbti`: 빌드된 이미지를 각각 `toby/mbti:embedded-db`와 `toby/mbti`로 태그 지정합니다.
- `-build-arg NODE_VERSION=20.15.1`: `NODE_VERSION`이라는 빌드 인수에 20.15.1 값을 전달해 Dockerfile 내에서 사용할 수 있도록 합니다.
- `f ./Dockerfile`: 현재 디렉토리의 Dockerfile을 명시적으로 지정합니다.
- `-pull`: 빌드 시 필요한 베이스 이미지를 항상 최신 버전으로 업데이트합니다.
- `∴` 현재 디렉토리를 빌드 컨텍스트로 사용합니다.

```
docker buildx build \  
> -t toby/mbti:embedded-db \  
> -t toby/mbti \  
> --build-arg NODE_VERSION=20.15.1 \  
> -f ./Dockerfile \  
> --pull \  
> .
```

01 Image Build

Docker image 확인

```
docker image ls
```

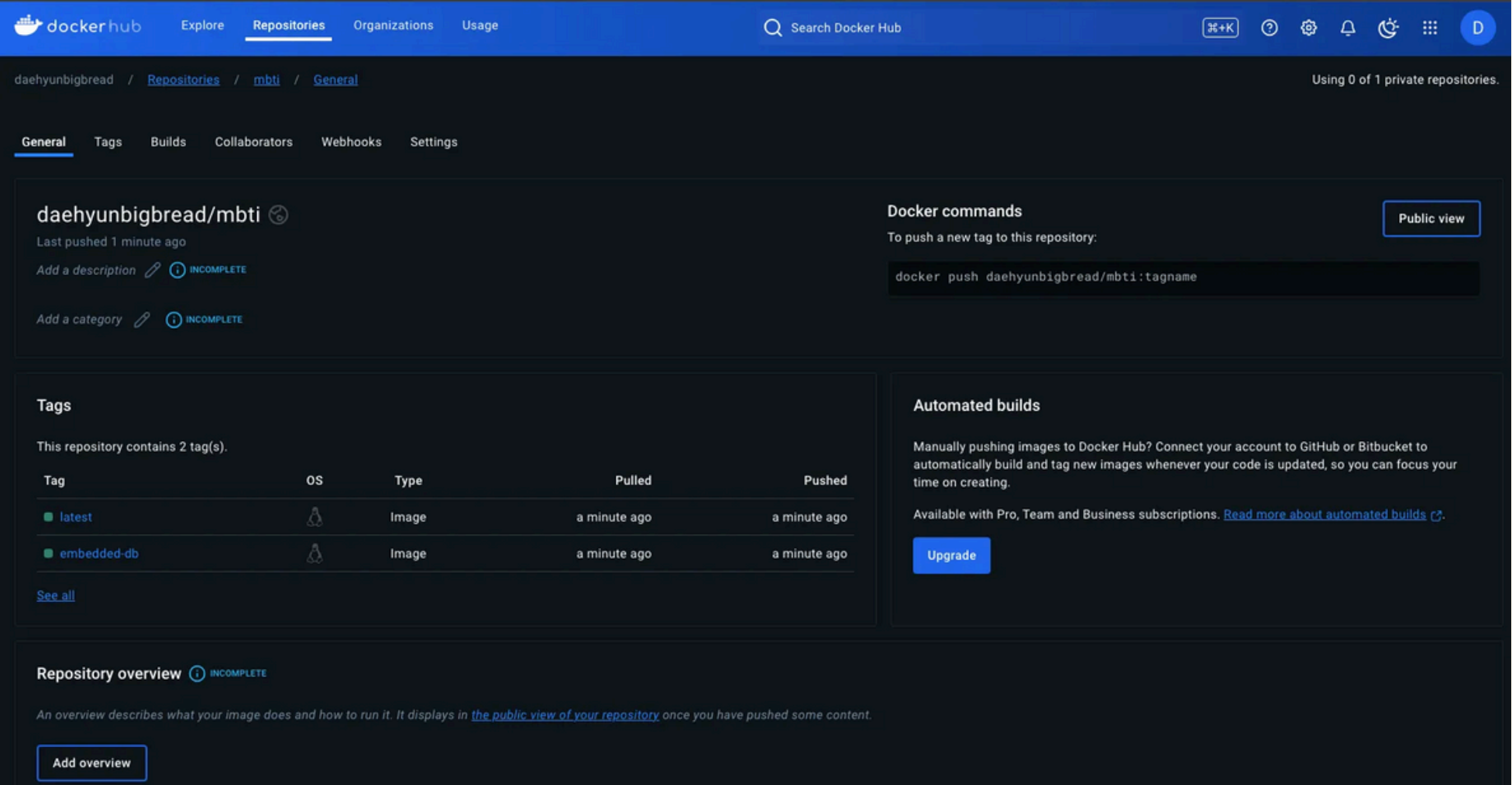
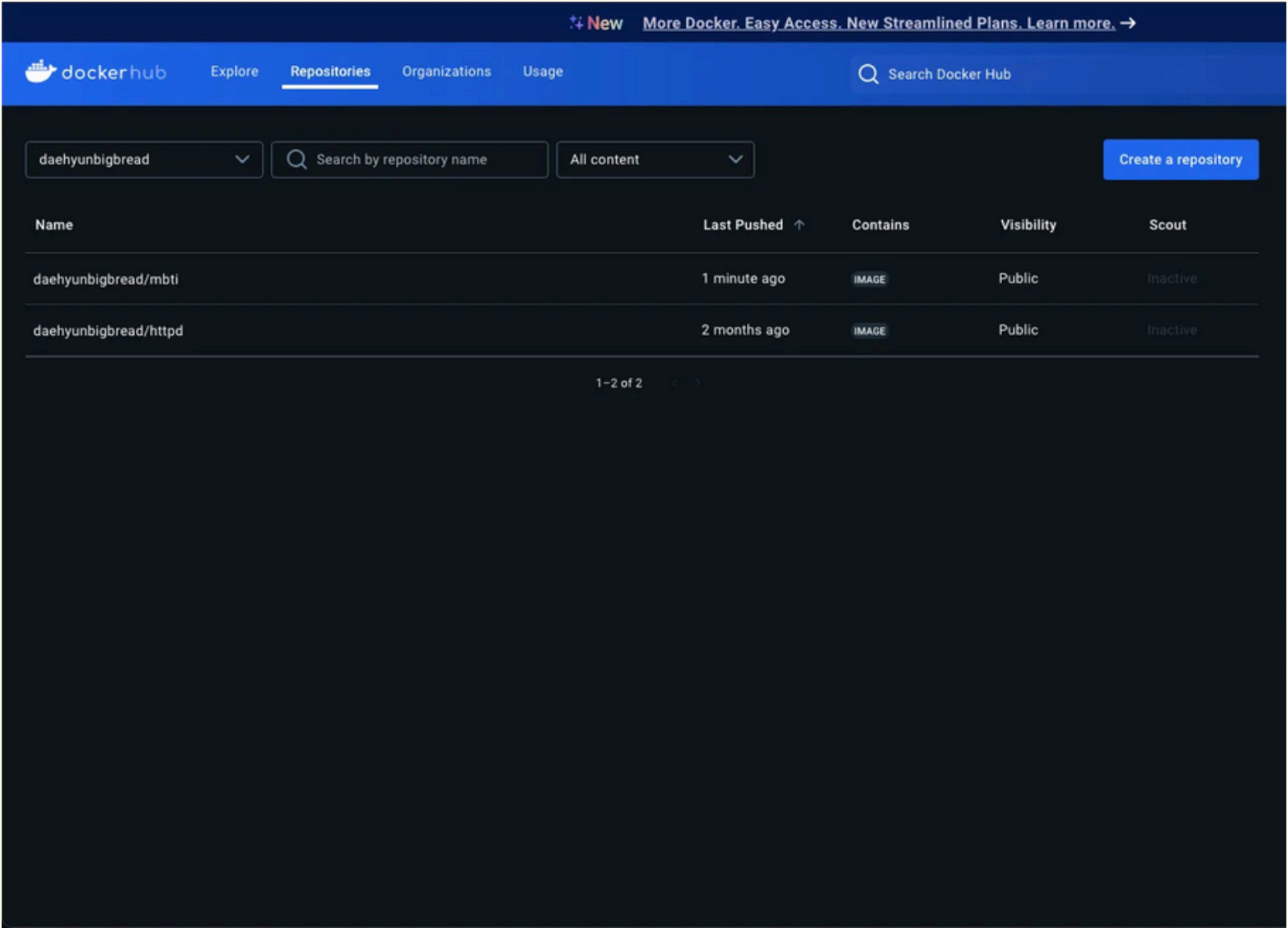
REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
toby/mbti	embedded-db	5dbed223aa10	About a minute ago	1.86GB
toby/mbti	latest	5dbed223aa10	About a minute ago	1.86GB

Docker Hub 배포

```
docker image push toby/mbti:embedded-db  
docker image push toby/mbti:latest
```

01 Image Build

Docker Hub 확인



Docker Container 실행

02 Container 실행

Container

- Docker 공식문서:
<https://docs.docker.com/reference/cli/docker/container/>
- **Container란?** : 앱을 실행하기 위해 격리된 경량 프로세스
- 프로세스 - 실행중인 프로그램 (메모장, 크롬브라우저 등..)
 - 메모리, 파일 시스템, 네트워크 등 컴퓨터의 다양한 자원 사용
 - 서로 간섭하지 않도록 메모리 수준에서 격리
- 컨테이너는 기본 프로세스보다 더 많은 자원을 격리
 - (파일시스템, 네트워크도 같이)
- **컨테이너가 필요한 이유?** → 하나의 컴퓨터(서버)에서도 격리된 환경 필요, 앱간 간섭 줄이기 & 특정 앱간 장애가 서로 영향을 주지 않기 위함.



02 Container 실행

Container (추가 개념)

- **요구사항에 따라 격리 수준을 다르게 할수도 있음** : host(컴퓨터)가 어떤 자원 층에서 격리 하냐에 따라 달라짐
 - → 메모리, 파일시스템, 네트워크, 라이브러리, OS, H/W 등등...
- **ex) 격리 기술**
 - **host os 수준**: 하이퍼바이저 → 단위는 가상머신 (vm, 높은 수준의 격리 제공, 보안상 이점. 단, os를 구성하기 위한 모든 실행요소가 필요. 무거워짐, 초기 구동속도 느림)
 - **app 수준**: Container → 단위: 컨테이너 (운영체제 관련 내용 없음, vm보다 가벼움 → 경량 process, 초기 구동속도 빠름 & 표준 기술로 자리잡음)
- **container는 process이기 때문에 정보를 전달할수 있는 수단이 있어야 함 → 이게 바로 image**

02 Container 실행

Container 실행



```
docker container run [OPTIONS] IMAGE[:TAG]
```

기본 실행 명령어

- `--name`: 컨테이너에 의미 있는 이름을 부여
- `-e`: 환경변수 설정 (KEY=VALUE): `--env-file` 옵션으로 환경변수를 파일로 선언해서 활용 가능
- `--rm`: 종료 시 컨테이너 파일 시스템을 자동 삭제
 - 컨테이너가 실행되는 과정에서 불필요한 정보, 데이터를 자동으로 삭제시켜줌
- `-d`: 백그라운드(Detached) 실행: 터미널을 다른 용도로도 사용하기 위한 목적

```
$ mbti-embedded-db>
docker container run \
  --name mbti \
  -e PORT=3000 \
  --rm \
  -d \
  learncodeit/mbti:embedded-db
```

Linux / MacOS

```
PS C:\mbti-embedded-db>
docker container run `
  --name mbti `
  -e PORT=3000 `
  --rm `
  -d `
  learncodeit/mbti:embedded-db
```

Windows

02 Container 실행

실행 예시

- 로그 보기: `docker container logs -f mbti`
- 쉘 접속: `docker container exec -it mbti bash`
- 중지: `docker container stop mbti`
- 강제 삭제: `docker container rm -f mbti`
- 전체 목록: `docker container ls -a` (Exited 컨테이너 포함)

정상 실행 시 `docker container ls` 에서 mbti 컨테이너가 보이고, STATUS가 Up 상태로 표시됩니다.

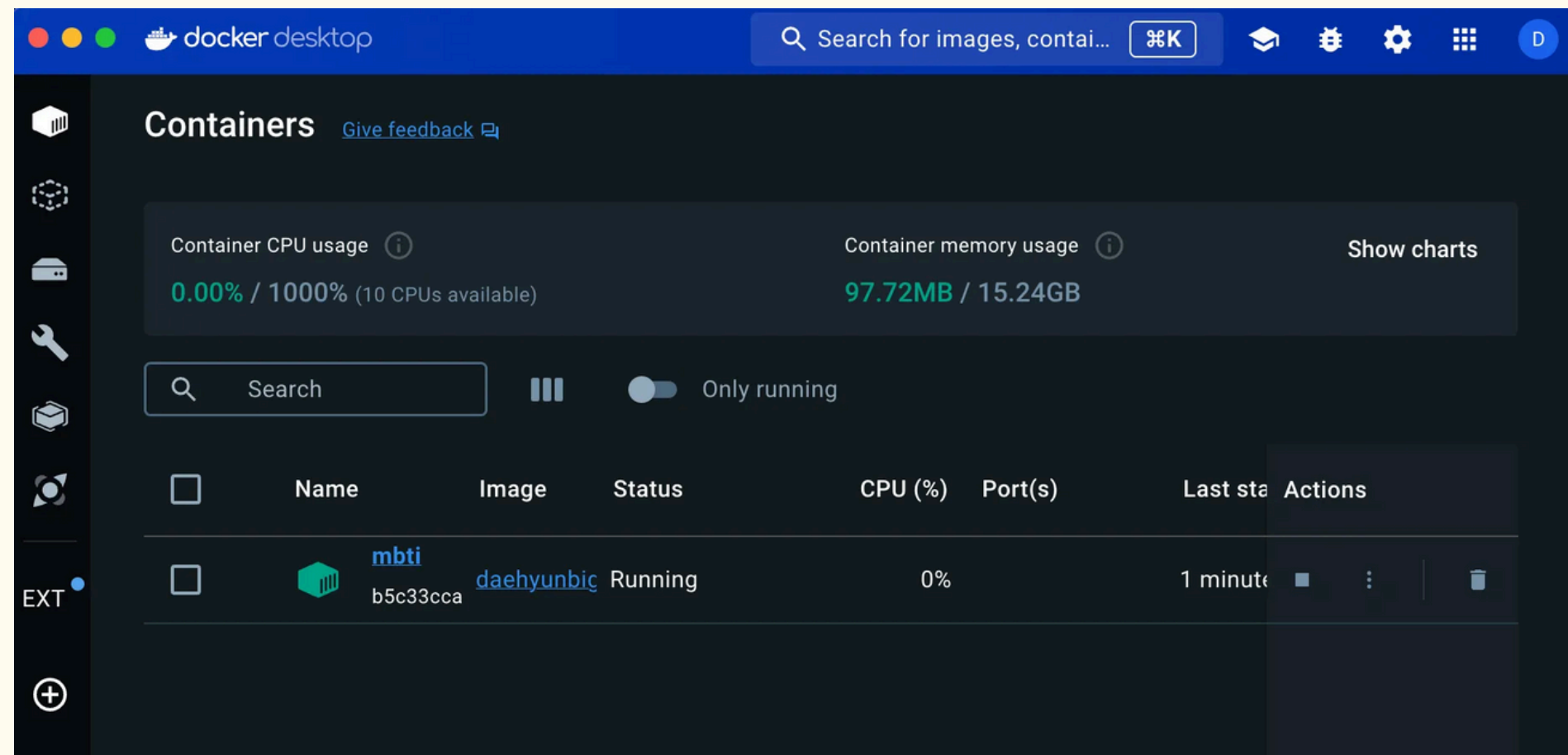
```
docker container run \
  --name mbti \
  -e PORT=3000 \
  --rm \
  -d \
  toby/mbti:embedded-db
```


02 Container 실행

실행 예시

```
docker container run \
--name mbti \
-e PORT=3000 \
--rm \
-d \
toby/mbti:embedded-db

b5c33cca7b0f895c4e3e1cce9e2ef7bc0c3059231b6c6a3fce0accf8f0a1b206
```



02 Container 실행

Container 확인, 관리

```
docker container ls
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
b5c33cca7b0f	toby/mbti:embedded-db	"npm run start"	12 minutes ago	Up 12 minutes		mbti

- container 내부의 파일 확인하고 싶으면?
 - docker container exec [옵션] [컨테이너] [실행할 명령어]

```
docker container exec mbti ls
```

```
docker container exec mbti ls /app
Dockerfile
next-env.d.ts
next.config.mjs
node_modules
package-lock.json
package.json
public
src
tsconfig.json
```

02 Container 실행

Container 확인, 관리

- Container를 터미널로 명령어로 접근하기 위해서? (컨테이너 내부에 shell-bash 실행)
 - -i & -t 옵션 실행 필요
 - -i: 키보드를 통해 입력하는 정보를 컨테이너에 전달하려는 옵션
 - -t: 컨테이너와 연결된 터미널을 실행하려는 옵션
 - 종료: exit



```
docker container exec -it mbti bash  
root@b5c33cca7b0f:/app#
```


02 Container 실행

Container 로그 확인

- -f: 실시간 로그 확인 목적
- curl을 활용한 요청 체크 (터미널창 따로 열어서 확인)



```
docker container logs -f mbti
```



```
docker container exec mbti \  
> curl localhost:3000/api/healthcheck
```



```
docker container logs -f mbti
```

```
> mbti-nextjs@0.1.0 start  
> next start
```

```
▲ Next.js 14.2.3
```

```
- Local:      http://localhost:3000
```

```
✓ Starting...
```

```
✓ Ready in 244ms
```

```
| GET | /api/healthcheck | Tue Jan 07 2025 07:35:11 GMT+0000 (Coordinated  
Universal Time) |
```

02 Container 실행

Container 로그 확인

- 실행중인 컨테이너 종료 명령어: **stop**, 실행중인 이미지는 **prune** 으로 입력해서 일괄 삭제 하면됨

```
docker container stop mbti
mbti

docker container ls
CONTAINER ID   IMAGE          COMMAND         CREATED        STATUS        PORTS        NAMES
```

- 만약 컨테이너가 종료만 되고 삭제가 되지 않았을때? 확인 방법
 - -rm 옵션을 컨테이너를 실행될때 입력하면 완료되면서 삭제됨. 만약 입력을 안했으면? **rm** 으로 삭제

```
docker container stop mbti
mbti

docker container ls -a
CONTAINER ID   IMAGE          COMMAND         CREATED        STATUS        PORTS        NAMES
aaf9f623ea59   daehyunbigbread/mbti:embedded-db   "npm run start"   27 seconds ago   Exited (1) 3 seconds ago             mbti

docker container rm mbti
mbti

docker container ls -a
CONTAINER ID   IMAGE          COMMAND         CREATED        STATUS        PORTS        NAMES
```

Docker Volume

03 Volume

Docker Volume

- Docker 공식문서:
<https://docs.docker.com/reference/cli/docker/volume/>
- **Volume?** : Docker가 관리하는 스토리지. 컨테이너와 관계없이 존재.
- 테이너의 내부와 연결된 외부의 디렉토리 (파일을 연결 - 마운트)
 - 마운트된 경로에 파일을 생성되면 연결된 경로에 파일이 생성
 - 다른 컨테이너에서도 연결 가능
- Docker Volume과 Container 연결: -v 옵션으로 진행
 - [볼륨 이름:컨테이너 내부의 마운트할 경로]

03 Volume

Docker Volume

- Docker 공식문서:
<https://docs.docker.com/reference/cli/docker/volume/>
- **Volume?** : Docker가 관리하는 스토리지. 컨테이너와 관계없이 존재.
- 컨테이너의 내부와 연결된 외부의 디렉토리 (파일을 연결 - 마운트)
 - 마운트된 경로에 파일을 생성되면 연결된 경로에 파일이 생성
 - 다른 컨테이너에서도 연결 가능
- Docker Volume과 Container 연결: -v 옵션으로 진행
 - [볼륨 이름:컨테이너 내부의 마운트할 경로]

실습용

- 이미지박스 (내부에 리눅스 명령어 사용가능하게 해주는 이미지) - 테스트 할때 명령어 사용 가능

```
docker image pull busybox
Using default tag: latest
latest: Pulling from library/busybox
559c60843878: Pull complete
Digest: sha256:2919d0172f7524b2d8df9e50066a682669e6d170ac0f6a49676d54358fe970b5
Status: Downloaded newer image for busybox:latest
docker.io/library/busybox:latest
```

- volume 생성 (볼륨 이름을 지정 안하면 랜덤한 문자열로 지정됨)

```
docker volume create busybox-volume
busybox-volume
```


03 Volume

Docker Volume

- 컨테이너 삭제시 볼륨 안에 있는 데이터들이 다 있을까?
 - 컨테이너 삭제후 재생성해서 실행해보면? 컨테이너는 삭제되어도 볼륨 안에 있는 데이터는 그대로 남아있는걸 볼수 있다.

```
docker container run \
> --name busybox \
> --rm \
> -v busybox-volume:/data \
> -it \
> busybox \
> sh
/ # cd /data/
/data # ls
box.log  busy.dat
```

- 그러면 연결을 끊는 방법은? 컨테이너 처럼 inspect 명령어 확인 후 가능
 - 이름 지정 안하면 랜덤한 문자열로 지정 (익명 볼륨)

```
docker container inspect busybox

.....

    "Mounts": [
      {
        "Type": "volume",
        "Name": "busybox-volume",
        "Source":
"/var/lib/docker/volumes/busybox-volume/_data",
        "Destination": "/data",
        "Driver": "local",
        "Mode": "z",
        "RW": true,
        "Propagation": ""
      }
    ],

.....
```

03 Volume

Docker Volume

- 만약 도커파일에 볼륨을 지정하려고 하면?
 - 볼륨 이름이 아니라 컨테이너 내부에 마운트할 디렉토리를 지정해주기.

```
# Node 설치. (이미지 관점 - 맨 아래 Layer (Base image))
# FROM 베이스 이미지
ARG NODE_VERSION
FROM node:${NODE_VERSION}

# 소스코드 다운로드
# COPY <복사할 경로> <붙여넣기를 할 경로>
COPY . /app

# 소스코드의 최상단 디렉토리로 이동
WORKDIR /app

# 소스코드를 실행할 때 필요한 파일 다운로드 (npm ci)
# 소스코드를 빌드 (npm run build)
RUN npm ci \
&& npm run build

# 환경변수 정의 (PORT)
ENV PORT=3000

# 마운트할 디렉토리 지정
VOLUME /data

# 소스코드 실행 (npm run start)
ENTRYPOINT ["npm", "run", "start"]
```

- 직접 호스트에 특정 경로를 지정해서 볼륨처럼 사용할수도 있음. (바인드 마운트)
- -v 옵션과 매우 유사: 옵션값에서 볼륨 이름 대신 호스트의 특정 경로 지정

```
docker container run \
> --rm \
> -it \
> -v /Users/daehyunkim/Downloads/mbti-embedded-db/host-
volume:/data \
> busybox \
> sh
/ # cd /data/
/data # touch box.log
/data # touch busy.dat
```

- 이렇게 생성할때 마다 생기는걸 볼 수 있다.

host-volume

box.log

busy.dat

03 과제

과제 1. Nest.js 서버 이미지 빌드

- 지난주 **과제로 제작한 Dockerfile로 Docker image Build**하기. (도커 이미지 빌드된 터미널창 캡처해서 제출)
- 빌드할 App: <https://github.com/hufs-pnp/25-Cloud-mbti>
 - Github에서 **본인 컴퓨터로 git clone**후, **clone한 repo directory 최상단에서 도커파일을 만들고 빌드**하세요.

과제 2. Next.js 서버 컨테이너 실행하기

- **다음의 조건에 부합하는 컨테이너를 실행**하세요. 컨테이너가 잘 실행되었다면, **실시간으로 뜨는 컨테이너 로그를 확인**해보세요.
- 컨테이너 로그를 확인하기 위해 **아래의 명령어를 입력해서 실행**해보세요. **아래와 같은 로그가 나와야 합니다.**
 - **터미널창 및 실행된 홈페이지 화면 캡처해서 제출**

1. 컨테이너 이름: mbti

2. 이미지: [계정명]/mbti:embedded-db

3. 환경 변수

- **PORT: 3000**

4. 컨테이너 종료 시 자동 삭제

5. 백그라운드 실행

- 컨테이너에 요청을 보내는 명령어(따로 터미널창 열고 실행하기)
 - **명령어**: `curl localhost:3000/api/healthcheck`

```
docker container logs -f mbti

> mbti-nextjs@0.1.0 start
> next start

  ▲ Next.js 14.2.3
  - Local:      http://localhost:3000

  ✓ Starting...
  ✓ Ready in 279ms
  | GET | /api/healthcheck | Tue Jan 07 2025 08:03:44 GMT+0000
  (Coordinated Universal Time) |
```


03 과제

제출 방식

- **과제1, 2: Notion Page에 넣어서 링크 전송**
 - Page Link 제출 하는곳: Cloud Squad Notion - Follow up System (5주차)
- **Due Date: 5/21 (수 19:30 까지)**
- 과제 하다가 오류 있으면 바로 DM으로 저에게 말해주세요! (오류가 있을수도 있습니다.... ㅈㅈ....)

다음 미팅

- **Squad Meeting Week 5**
 - 일시: 05.21 (수) 20:00 ~ 21:00
 - 예정시간: 50~60분
 - 내용 Docker Part.3 (Docker 네트워크, Docker Compose)